

Prague University of Economics and Business
Faculty of Informatics and Statistics



**Utilization of machine learning
methods for predicting the outcomes
of football matches**

BACHELOR THESIS

Study program: Data Analytics

Author: Martin Fedorík

Supervisor: Ing. Ondřej Sokol, Ph.D.

Prague, June 2024

Acknowledgements

I want to express my gratitude to my thesis advisor, Ing. Ondřej Sokol, Ph.D., for his invaluable guidance and support throughout writing this thesis.

Abstract

This bachelor's thesis investigates the use of machine learning to predict football match outcomes, aiming to achieve high predictive accuracy and compare it with bookmakers' accuracy. A secondary goal was to evaluate the potential profitability of the models. We utilized a novel comprehensive dataset with detailed statistics, including expected goals (xG), to enhance predictive accuracy. The dataset covered seven English Premier League (EPL) seasons, with the 2023/2024 season as the test set. Extensive feature engineering was conducted, resulting in 134 features, of which 52 were selected using recursive feature elimination with XGBoost. Hyperparameters for XGBoost and Random Forest models were optimized through time-based cross-validation and random search. The models achieved an accuracy of 58.4% (Random Forest) and 58.6% (XGBoost) on 370 matches from the 2023/2024 EPL season, closely matching the bookmakers' accuracy of 58.9%. A significant shortcoming was the models' inability to predict draws, likely due to class imbalance and the use of accuracy as the evaluation metric for feature selection and hyperparameter tuning. Theoretical profitability was assessed by betting a constant amount based on predictions, but it was not statistically significant at the 5% level. Future research could incorporate player-level features, dive deeper into the problem of draw predictions and explore more sophisticated betting strategies to achieve statistically significant profits.

Keywords

machine learning, football, match prediction, betting, sports

Abstrakt

Tato bakalářská práce zkoumá využití strojového učení k predikci výsledků fotbalových zápasů s cílem dosáhnout vysoké prediktivní správnosti a porovnat ji se správností sázkových kanceláří. Sekundárním cílem bylo zhodnotit potenciální ziskovost modelů. Byla použita nová komplexní datová sada s podrobnými statistikami, včetně očekávaných gólů (xG), k zvýšení prediktivní správnosti. Datová sada zahrnovala sedm sezón anglické Premier League (EPL) s tím, že sezóna 2023/2024 sloužila jako testovací sada. Bylo provedeno rozsáhlé generování příznaků, které vedlo k 134 příznakům, z nichž bylo 52 vybráno pomocí rekurzivní eliminace příznaků s XGBoost. Hyperparametry pro modely XGBoost a Random Forest byly optimalizovány pomocí časově založené křížové validace a náhodného vyhledávání. Modely dosáhly správnosti 58.4% (Random Forest) a 58.6% (XGBoost) na 370 zápasech sezóny EPL 2023/2024, což se blížilo správnosti sázkových kanceláří 58.9%. Významnou slabinou byla neschopnost modelů předpovídat remízy, pravděpodobně kvůli nevyváženosti tříd a použití správnosti jako hodnotící metriky pro výběr příznaků a ladění hyperparametrů. Teoretická ziskovost byla posuzována sázením konstantní částky na základě predikcí, ale nebyla statisticky významná na 5% hladině významnosti. Budoucí výzkum by mohl zahrnout příznaky na úrovni hráčů, hlouběji se podívat na problematiku predikce remíz a prozkoumat sofistikovanější sázkové strategie k dosažení statisticky významných zisků.

Klíčová slova

strojové učení, fotbal, predikce zápasů, sázení, sport

Contents

Introduction	6
1 Literature review	7
2 Machine Learning	9
2.1 Categories of machine learning	9
2.2 Decision Trees	9
2.3 Random Forest	10
2.4 Ensemble Learning	10
2.5 Boosting	11
2.5.1 Gradient Boosting	11
2.6 Feature Engineering	13
2.7 Feature Selection	13
2.8 Model validation	13
2.8.1 Data leakage	14
2.8.2 Hyperparameter tuning	14
2.8.3 Cross validation	15
2.9 Model Performance Evaluation	16
3 Data and feature engineering	18
3.1 Dataset	18
3.2 Feature engineering	20
3.2.1 Performance metrics	20
3.2.2 Historical performance	21
3.2.3 Differences and ratios	22
4 Modeling	23
4.1 Experiment design	23
4.2 Feature selection	23
4.3 Random forest	23
4.4 XGBoost	26
5 Comparison of prediction methods	28
6 Betting simulation	29
Conclusion	31
Bibliography	32
A Full form	36

Introduction

Football is one of the most popular sports in the world, and its popularity has been on the rise in recent years. This increase in popularity has naturally led to more people being interested in predicting the outcome of football matches, either as a hobby or as a means to make money by betting on the matches through bookmakers or betting exchanges. Many people might bet purely on their favourite teams without considering any other option, while others might try to examine the teams' recent performance. Even for those who try to predict the result based on statistics, personal preferences and feelings still play a big role in the bettor's decision-making.

In today's data age, much more detailed data is collected from football matches than ever with the help of computer vision and artificial intelligence. This is especially true for the most popular football leagues in the world. Bookmakers are able to leverage vast amounts of detailed data about past matches to determine their odds as accurately as possible. The bookmakers further adjust these calculated odds to include a margin and minimize risk.

The motivation for this bachelor's thesis comes from a personal interest in machine learning and sports betting. The main goal is to utilize modern machine learning methods and publicly available data to predict the outcome of football matches and compare the performance against the bookmakers' predictions and other naive prediction methods. A secondary goal is to evaluate whether betting based on our model's predictions would lead to a profit over the long term.

This work is divided into six sections. In the first one, we review the existing literature about football match prediction. The second part describes the theory behind the machine learning models and methods used. The third chapter contains a description and structure of the dataset and a section where the feature engineering process is described. In the fourth and fifth sections, we describe the experiment design and modelling procedure and compare the performance of different prediction methods. The last section evaluates the potential profitability of various methods if we place bets according to various prediction methods.

1. Literature review

A large amount of research exists that deals with the problem of predicting football matches, but more often than not, it applies statistical and regression models instead of purely machine learning methods. In the existing research, the majority of the papers are based on the same dataset from ('FootballData', 2024), which only includes very high-level statistics from matches. This might limit the potential performance of the classification models. The predictive performance is most often measured in terms of accuracy.

Gomes et al. (2015) present the implementation of a decision support system that uses machine learning models, more specifically Naive Bayes, Decision Trees, and Support Vector Machines, in order to predict football match outcomes using Premier League data from the 2000/2001 season up to the 2012/2013 season. The best model was the Support Vector Machine, which achieved an accuracy of 50.8% on approximately 33% of out-of-sample data. The main features used for prediction were average goals, shots, and shots on target for both the home and away teams in the previous matches before a specific match. The authors also created features capturing the previous wins and losses of the most recent games.

Prasetio and Harlili (2016) used logistic regression for binary classification of either home or away win. Covariates for the logistic regression were offence and defence ratings from the video game FIFA. The model achieved a predictive accuracy of 69.5% in the 2015/2016 season of the English Premier League. Still, it's important to note that this result is not directly comparable to other results, as the draws were excluded from the evaluation, which might give the impression that the model is very accurate. However, in the 2015/2016 English Premier League season, 107 draws out of 380 matches were played ('WhoScored-PremierLeague', 2024), which is 28.2% of matches played. Assuming that this model would mispredict the 107 draw matches, the accuracy, including the draws, would be 50%

Shin and Gasparyan (2014) demonstrated a novel way to predict the outcomes of real soccer matches by using virtual features from the video game FIFA. Three models were used for the binary classification of each outcome: Home or non-home, draw or non-draw, and away or non-away. The models trained on virtual data achieved an accuracy of 78%, 80%, and 78% for Home, Draw, and Away outcomes, respectively. The author did not specify the season for which predictions were made, and the league was the Spanish La Liga. These accuracy metrics are a bit misleading compared to the rest of the research because of imbalanced classes in the binary classification models. In the case of draws, if we assume 30% of the matches in the entire dataset are draws, we could achieve a 70% accuracy by always predicting non-draws, simply due to the class imbalance of draws and non-draws. Nevertheless, using multiple models for binary classification of each outcome instead of a single model for multiclass classification is an interesting approach worth considering. According to this publication, including FIFA video game ratings of players and teams in combination with real-world statistics about matches might yield better results than relying purely on real-world data.

Alfredo and Isa (2019) achieved an accuracy of 68.6% using a random forest algorithm, but upon closer inspection, we can see that this publication has a critical methodological issue, and that is the usage of match statistics to predict the outcome of the match itself. The model uses features such as the number of shots, shots on target, and half-time goals, none of which are known before the game occurs. This highlights that it is necessary to perform careful validation when creating and using features to ensure we do not use information that could have been available only during or after the match itself.

2. Machine Learning

Machine learning is a subfield of Artificial Intelligence (AI) that studies algorithms used to make automatic decisions based on the data provided to the algorithm (Mahesh, 2019).

2.1 Categories of machine learning

Supervised machine learning algorithms are used to learn the patterns and numerical relationships from data that determine the target variable of interest. The goal is to train the algorithm's parameters using the given training data so that the algorithm can further be applied to unseen data to generate predictions for the target variable. In classification tasks, the target variable refers to a discrete set of unique values or labels. In regression tasks, the variable can be either a discrete or continuous numeric value (Liu & Wu, 2012).

Unsupervised learning deals with problems where our goal is to find patterns within the data itself that might or might not be apparent at first sight. A common task is finding clusters in a high-dimensional space, which is very hard, if not impossible, to do manually (Naeem et al., 2023).

2.2 Decision Trees

Decision trees are a popular machine learning tool used for classification tasks, where the objective is to predict a categorical variable, and regression tasks that require the prediction of a continuous variable. The core working principle behind decision trees is partitioning of the data into smaller subsets based on specific data attributes. A decision tree is composed of nodes, where each node represents a splitting attribute. There are usually two or possibly more edges coming from a single node, where each edge represents an attribute's value based on which the subset is determined. At the lowest level of the tree are leaves, which contain the final subset of data after partitioning. Decision tree inducers are algorithms that are used to construct decision trees. They usually construct the trees in a top-down fashion, starting at the root node and determining the attribute and its value, which minimizes the prediction error relative to some target value of the dataset. However, different objectives can also be used, such as minimizing the number of nodes or combining multiple approaches (Rokach & Maimon, 2005).

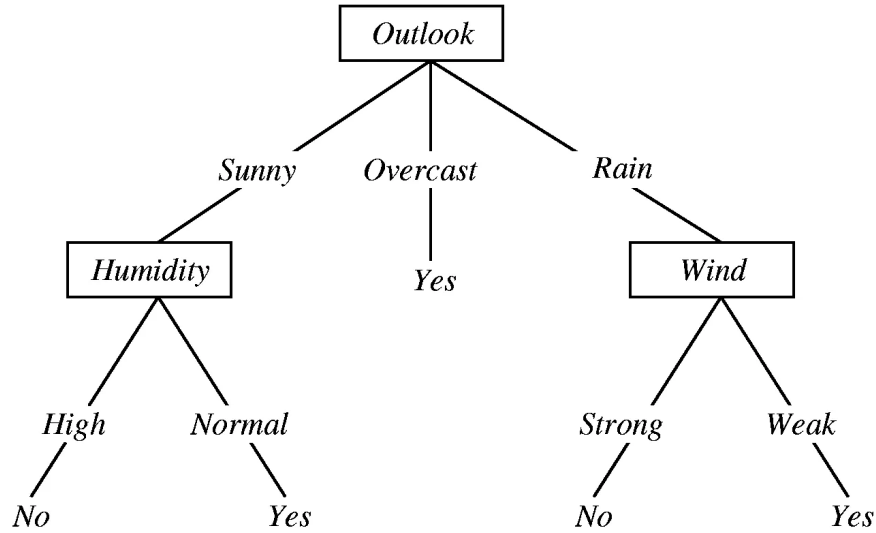


Figure 2.1: Decision tree for rain forecasting (Yadav, 2019).

2.3 Random Forest

Random forest is an ensemble supervised machine learning algorithm introduced by Breiman (2001). It utilizes multiple decision trees and implements various techniques to reduce overfitting and increase the generalization performance on unseen data. When building a single decision tree, we would need to build a very deep tree to capture the complexities within the dataset, leading to poor generalization ability. Random forests overcome this by constructing multiple, usually simpler trees, using subsets of the original data. The subsets are generated at training time using sampling with replacement from the original dataset, also called bootstrapping, so they can be used to train multiple diverse trees. Additionally, a random subset of the original features is selected for each subset. Each tree is then trained on its respective data subset, and the predictions from this algorithm are generated either by majority voting for classification tasks or for regression tasks; we take the arithmetic mean of predictions from all trees (Breiman, 2001).

2.4 Ensemble Learning

A set of multiple different machine learning algorithms (also called learners) that are used to predict the same target variable is called an ensemble. The ensemble learning paradigm aims to achieve greater overall predictive power by combining multiple learners. Ensembles can be broadly classified into two main categories: generative and non-generative methods. Generative methods focus on creating weak learners by manipulating the dataset and how learners are trained to create diverse models with varying accuracy. Non-generative methods focus on optimally combining the outputs of multiple already trained and optimized models to achieve greater accuracy than any single one of them. Some popular methods of combining outputs of multiple models are majority voting and weighted voting (Valentini & Masulli,

2002).

2.5 Boosting

Boosting is an ensemble machine-learning technique that combines multiple weak machine-learning algorithms for a more accurate predictor. It was introduced by Schapire (1990). During training, we first start with a weak learner that is slightly better than random guessing, which could be, for example, a simple decision tree with one split, also known as a stump, and get the predictions. In the next iteration of learning, after retrieving the predictions from the first weak learner, we look at the incorrect predictions made by the first learner and increase the weight for those observations, while for the correct predictions, we decrease the weight. After that, we add another weak learner focusing on incorrectly predicted instances. We do this iteratively until our accuracy is satisfactory. The final prediction is a combination of all the weak learners' predictions, usually weighing the predictions by the respective learner's accuracy (Schapire, 1990, 2003).

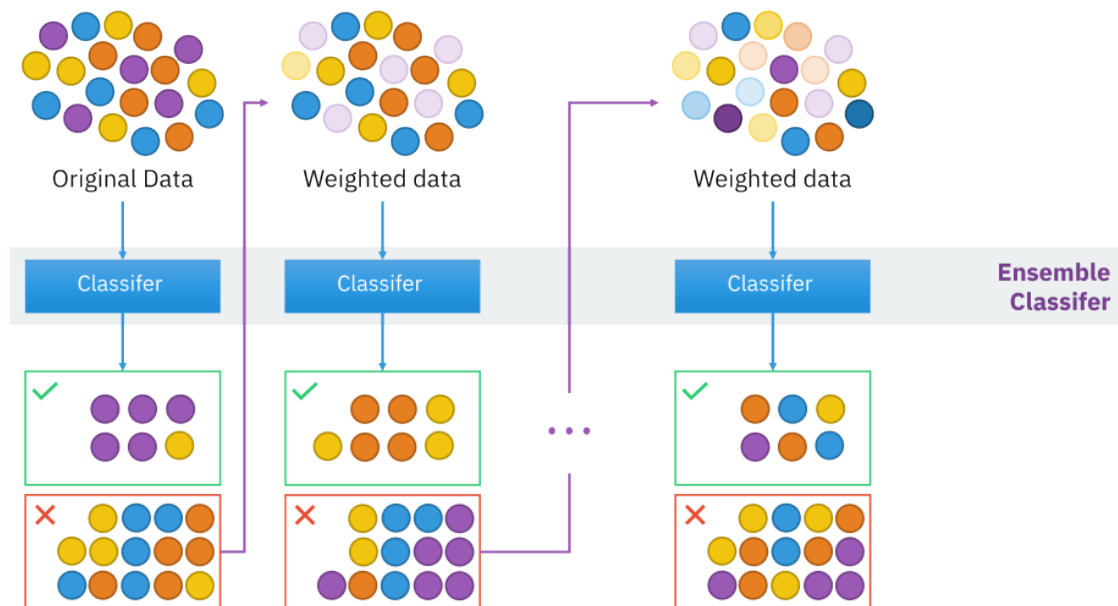


Figure 2.2: Boosting visualization for classification (Sirakorn, 2020).

2.5.1 Gradient Boosting

Gradient boosting is an advanced boosting approach, which was introduced by Friedman (2001). The critical distinction is that the target variable for training weak learners is the pseudo-residual (or pseudo-response). In classical boosting, the target variables are the residuals. The objective is to estimate the function $F(\mathbf{x})$ that minimizes the loss function

$L(y, F(\mathbf{x}))$, where $F(\mathbf{x})$ represents the functional relationship between the input variables and the target variable:

$$\hat{F}(\mathbf{x}) = \arg \min_F \mathbb{E}[L(y, F(\mathbf{x}))].$$

Gradient boosting iteratively improves the ensemble by adding weak learners $h_m(\mathbf{x})$, each trained to predict the pseudo-residuals from the previous iteration. At each step m , the model is updated as follows:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x}),$$

where ν is a learning rate that controls the contribution of each weak learner. The pseudo-residuals $\tilde{y}_i$ are computed as the negative gradient of the loss function with respect to the model's prediction:

$$\tilde{y}_i = - \left[\frac{\partial L(y_i, F_{m-1}(\mathbf{x}_i))}{\partial F_{m-1}(\mathbf{x}_i)} \right].$$

These pseudo-residuals $\tilde{y}_i$ serve as the target variable for training the next weak learner $h_m(\mathbf{x})$. This approach allows the model to focus on the areas where it performs poorly, effectively reducing the overall prediction error in subsequent iterations. One of the main advantages of gradient boosting is its flexibility. It can be used with various types of weak learners, such as decision trees, and it can optimize different loss functions, making it suitable for both regression and classification problems. The learning rate ν plays a big role in balancing the trade-off between the model's performance and its generalizability. A lower learning rate typically requires more iterations but can lead to a more robust model. Regularization techniques can be incorporated into gradient boosting to prevent overfitting. Common regularization methods include limiting the depth of the weak learners in case of using decision trees as weak learners, subsampling the data, and adding noise to the data or the pseudo-residuals. These techniques enhance the model's ability to generalize to new data (Friedman, 2001).

eXtreme Gradient Boosting(XGBoost) is an optimized framework for gradient tree boosting. It was initially released in 2014 and quickly gained popularity due to being used in many winning solutions of various Kaggle competitions ('XGBoost', 2024). The technicalities behind XGBoost are further described in the paper by Chen and Guestrin (2016). Key features of XGBoost include regularization, parallel tree building, shrinkage, which reduces overfitting, column subsampling, and various other methods used to optimize the computational efficiency of gradient tree boosting (Chen & Guestrin, 2016). The magnitude of L1 and L2 regularization is set using the *alpha* and *lambda* hyperparameters ('XGBoost Parameters', 2024).

2.6 Feature Engineering

Feature engineering is a key process in machine learning and predictive modelling that uses raw data to create features for the machine learning model. The goal is to create features that can capture more complex information and relationships from the dataset and thus improve the performance of the models trained on it. Engineering new features is a manual, time-consuming task requiring extensive domain knowledge to create features that will help the model capture the complex relationships between raw data and the target variable (Rawat & Khemchandani, 2019). The most common feature engineering techniques include computing differences between features, ratios between features, and various count forms. An example of a count feature in our task can be the number of consecutive wins in the last five games (Heaton, 2016).

In the task of football match prediction, feature engineering is even more important, and the reason for that is that our goal is to predict the outcome of a football match before it starts. This means that features that capture the teams' past performance based on historical match and player statistics have to be created, as naturally, we cannot use a match's statistics to predict its outcome as the match hasn't occurred yet.¹

2.7 Feature Selection

Feature selection is a machine learning technique that addresses the high dimensionality problem by selecting relevant features and eliminating irrelevant, redundant and noisy ones. This process aims to enhance model performance without transforming the original features, making the models more interpretable and generalizable (Bouchlaghem et al., 2022).

Recursive Feature Elimination with Cross Validation is a backward feature selection technique that eliminates the least important features in each iteration until we have one feature left. We start with a complete feature set and iteratively remove the least important feature based on their feature importance scores, which are returned by the estimator used for evaluating the tested feature sets. In each iteration, the model/estimator is trained on the training folds and evaluated on validation folds to obtain the set of features that maximizes a predefined validation metric (Awad & Fraihat, 2023; 'RFECV', 2024).

2.8 Model validation

Model validation is a crucial and essential step in the process of training machine learning models. The purpose of validation is to assess the performance and generalizability of a model

¹Unless we are predicting the outcome of live matches, and in that case, we would need real-time data.

on unseen data during training. The validation process can also include the optimization of hyperparameters of the specific model (Maleki et al., 2020).

2.8.1 Data leakage

Data leakage is a common problem in machine learning, and it refers to a situation when some data or information from the validation or test set is unintentionally included in the training dataset. If data leakage occurs, the model performance on the validation/test set might be overly optimistic because the model was partially trained on the validation/test data or the training data included some form of information from test set (Nassar, 2023). In our case, it's very important to keep the risk of data leakage in mind, as football matches are time-dependent. When partitioning our dataset into train/validation/test sets, this must be considered. Random sampling cannot be used, as with random sampling, it could happen that the model would be trained on future matches, and the test set would contain matches from the past, violating the temporal dependence of individual matches. An example of data leakage could be estimating a team's current performance by taking the mean performance over the previous five games and then splitting the dataset randomly. With random sampling, the games from which we would calculate the mean could end up in the test set, and the training set would then contain information from the matches in the test set, as it would be included in the training mean.

2.8.2 Hyperparameter tuning

Hyperparameter of a model is a configuration option that is set before the training of a model, and can be used to affect the way how the model learns from data (Simon et al., 2023). An example hyperparameter for random forest could be maximum number of trees in the ensemble (Breiman, 2001).

Hyperparameter tuning refers to the process of finding a combination of hyperparameters for a specific model that achieves the best performance. A separate validation set or cross-validation is necessary for evaluating the combinations of hyperparameters. Various models have different kinds of hyperparameters that make up the hyperparameter space. The performance of a model can be significantly improved with an extensive search of the predefined hyperparameter space (Simon et al., 2023). The main drawbacks of this search are that it is very computationally intensive and it has to be accompanied by robust model validation techniques to avoid overfitting the hyperparameters to the validation data, which can happen when there are many hyperparameters to tune or our validation sets are not created by random sampling (Levesque, 2018). Many methods for hyperparameter search exist, and some of the most often used ones are random search, in which we randomly select combinations to evaluate from the search space, or grid search, which represents an exhaustive evaluation of all the hyperparameter combinations in the predefined hyperparameter space. Some more

advanced methods include Bayesian optimization, meta-learning and bandit-based methods (Simon et al., 2023).

2.8.3 Cross validation

Cross-validation is a model validation technique used to estimate the model’s generalization performance on unseen data and avoid overfitting. It is also often used and necessary for the hyperparameter tuning of models and the feature selection process. In a typical cross-validation setting, we partition the learning set, which is used to train and evaluate model performance, into a training set(s) and testing(validation) set(s) (Berrar, 2018). The testing set(s), in this case, is also called validation set(s) if the original dataset was partitioned into a learning set and test set(used for final evaluation). There are numerous cross-validation methods, each tailored to specific use cases. The choice of validation method largely depends on the computational limitations and structure of the dataset.

One of the most commonly used is called k -fold cross-validation, in which we randomly partition the learning set into k similarly sized subsets. After that, the model is trained on $k - 1$ subsets and evaluated on the remaining validation subset, which was not used in training. We do this k times, each time selecting a different subset used for the validation. The average of the performance across the validation subsets is called the cross-validation performance (Berrar, 2018).

In our case, using classical k -fold cross-validation is not possible, as it would violate the time dependency of football matches, and we could end up with past matches in the testing set and future matches in training(see Sec. 2.8.1). A cross-validation splitting technique must be used that respects this time dependence. One such technique is called prequential(Preq-bl), also called time series split cross-validation. The idea is to split the data into multiple blocks, ensuring each block preserves the temporal order of the data. In each cross-validation iteration, one block is used as the validation set, while the remaining blocks before the validation set(in time) are used for training (Cerqueira et al., 2022). This process can be seen visualized in Figure 2.3, where the blue colour represents the training set and the red colour shows the validation set; the x axis, in this case, refers to the index of ordered observations.

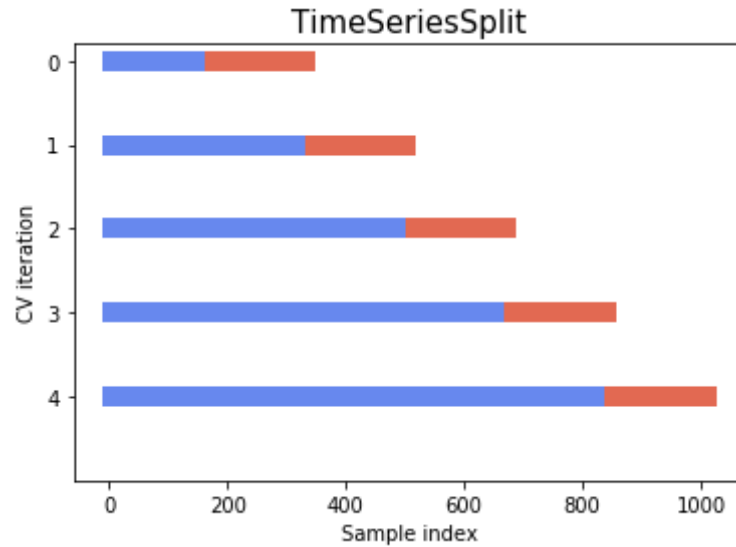


Figure 2.3: Time series split cross validation (Packt, 2019).

2.9 Model Performance Evaluation

Model performance evaluation is one of the final steps in solving a task using machine learning. To evaluate the generalizability of a model, the trained model's performance is measured on the part of the dataset that was not used during the training or hyperparameter tuning process. This final out-of-sample evaluation ensures that we obtain an unbiased estimate of the model's performance on unseen data. It might also be useful to evaluate the model on the training data and compare the error on the training vs. test (validation) set, where a large discrepancy might indicate an overfitting problem (Maleki et al., 2020).

Classification metrics are essential for evaluating the performance of a classification model. These metrics enable us to measure the performance of the model across different aspects of the classification task. The choice of an evaluation metric greatly depends on the goal of the task being solved.

Accuracy is the ratio of correctly predicted instances to the total instances. (Hossin & M.N, 2015) It is one of the most straightforward metrics to interpret, but it can be misleading in the case of imbalanced datasets, as was the case in (Shin & Gasparyan, 2014). Accuracy can be calculated in the following way:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}.$$

Precision also known as Positive Predictive Value, measures the proportion of true positive predictions out of the total predicted positives (Hossin & M.N, 2015). Precision for a binary

classifier is defined as follows:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}.$$

Recall also known as Sensitivity or True Positive Rate, measures the proportion of true positive predictions out of the actual positives (Hossin & M.N, 2015). Recall for a binary classifier can be calculated as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}.$$

F1 Score is the harmonic mean of precision and recall. It provides a balance between precision and recall and is useful when we need to account for both false positives and false negatives (Hossin & M.N, 2015). The F1 score is calculated in the following way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Confusion Matrix is a table that visualizes the performance of a classification model by displaying the true positives, false positives, true negatives, and false negatives for all classes. It provides a comprehensive view of how well the model performs across different categories and can help us identify shortcomings of the model (Hossin & M.N, 2015). An example of a confusion matrix for a binary classifier can be seen in Table 2.1.

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Table 2.1: Confusion matrix (Author's calculation)

3. Data and feature engineering

3.1 Dataset

The dataset contains data about matches played in the English Premier League (EPL) in the seasons from 2017/2018 up to 2023/2024. In total, 2660 matches were played(380 per season), but in our dataset, data for four matches is missing, so they are excluded. The data comes from multiple online public sources, was obtained by web scraping, and then joined together based on dates and team names. The primary source of the detailed match statistics was the webpage www.whoscored.com ('WhoScored', 2024). Historical betting odds were provided by www.football-data.co.uk ('FootballData', 2024), and a more advanced match performance metric called expectedGoals(xG) was supplied by www.footystats.org ('FootyStats', 2024). The merged dataset has a total of 74 attributes, described in Table 3.1.

The EPL is contested by 20 clubs each season, with clubs being relegated and promoted between the English Football League(EFL) and the EPL. There are 380 matches played each season; each team plays thirty-eight matches and plays against every other team twice, once at home and once away ('Premier League', 2024).

expectedGoals(xG) is an advanced metric that is used to measure the quality of shots based on the likelihood that they will be converted to a goal. It is determined using a statistical/-machine learning model that uses information about past shots. An important note is that as the xG measures the quality of shots in a game, it is calculated after the match occurs. The metric is described in more detail by Whitmore (2023).

Table 3.1: Description of all attributes in the dataset (Author's calculation)

Column	Value type	Description
date	Date	Date of the match
season	String	Season of the match
team_away	String	Away team name
result	String	Result of the match
team_home	String	Home team name
aerialsTotal_away	Integer	Total aerial duels by the away team
aerialsTotal_home	Integer	Total aerial duels by the home team
aerialsWon_away	Integer	Aerial duels won by the away team
aerialsWon_home	Integer	Aerial duels won by the home team
averageAge_away	Float	Average age of the away team players
averageAge_home	Float	Average age of the home team players
clearances_away	Integer	Clearances by the away team
clearances_home	Integer	Clearances by the home team
cornersAccurate_away	Integer	Accurate corners by the away team

Column	Value type	Description
cornersAccurate_home	Integer	Accurate corners by the home team
cornersTotal_away	Integer	Total corners by the away team
cornersTotal_home	Integer	Total corners by the home team
defensiveAerials_away	Integer	Defensive aerial duels by the away team
defensiveAerials_home	Integer	Defensive aerial duels by the home team
dispossessed_away	Integer	Times the away team was dispossessed
dispossessed_home	Integer	Times the home team was dispossessed
dribbledPast_away	Integer	Times away team players were dribbled past
dribbledPast_home	Integer	Times home team players were dribbled past
dribblesAttempted_away	Integer	Dribbles attempted by the away team
dribblesAttempted_home	Integer	Dribbles attempted by the home team
dribblesLost_away	Integer	Dribbles lost by the away team
dribblesLost_home	Integer	Dribbles lost by the home team
dribblesWon_away	Integer	Dribbles won by the away team
dribblesWon_home	Integer	Dribbles won by the home team
foulsCommitted_away	Integer	Fouls committed by the away team
foulsCommitted_home	Integer	Fouls committed by the home team
goals_away	Integer	Goals scored by the away team
goals_home	Integer	Goals scored by the home team
interceptions_away	Integer	Interceptions by the away team
interceptions_home	Integer	Interceptions by the home team
offensiveAerials_away	Integer	Offensive aerial duels by the away team
offensiveAerials_home	Integer	Offensive aerial duels by the home team
offsidesCaught_away	Integer	Offsides caught by the away team
offsidesCaught_home	Integer	Offsides caught by the home team
passesAccurate_away	Integer	Accurate passes by the away team
passesAccurate_home	Integer	Accurate passes by the home team
passesKey_away	Integer	Key passes by the away team
passesKey_home	Integer	Key passes by the home team
passesTotal_away	Integer	Total passes by the away team
passesTotal_home	Integer	Total passes by the home team
possession_away	Float	Possession percentage of the away team
possession_home	Float	Possession percentage of the home team
rating_away	Float	Rating of the away team
rating_home	Float	Rating of the home team
shotsBlocked_away	Integer	Shots blocked by the away team
shotsBlocked_home	Integer	Shots blocked by the home team
shotsOffTarget_away	Integer	Shots off target by the away team
shotsOffTarget_home	Integer	Shots off target by the home team
shotsOnTarget_away	Integer	Shots on target by the away team
shotsOnTarget_home	Integer	Shots on target by the home team

Column	Value type	Description
shotsTotal_away	Integer	Total shots by the away team
shotsTotal_home	Integer	Total shots by the home team
tackleSuccessful_away	Integer	Successful tackles by the away team
tackleSuccessful_home	Integer	Successful tackles by the home team
tackleUnsuccessful_away	Integer	Unsuccessful tackles by the away team
tackleUnsuccessful_home	Integer	Unsuccessful tackles by the home team
tacklesTotal_away	Integer	Total tackles by the away team
tacklesTotal_home	Integer	Total tackles by the home team
throwInsAccurate_away	Integer	Accurate throw-ins by the away team
throwInsAccurate_home	Integer	Accurate throw-ins by the home team
throwInsTotal_away	Integer	Total throw-ins by the away team
throwInsTotal_home	Integer	Total throw-ins by the home team
touches_away	Integer	Total touches by the away team
touches_home	Integer	Total touches by the home team
B365H	Float	Bet365 odds for home team win
B365D	Float	Bet365 odds for a draw
B365A	Float	Bet365 odds for away team win
xG_home	Float	Expected goals for the home team
xG_away	Float	Expected goals for the away team

3.2 Feature engineering

3.2.1 Performance metrics

The key idea behind these features is to transform raw football match statistics into more meaningful performance metrics that better quantify a team's strengths and weaknesses. By calculating success rates, accuracies, and percentages, we allow the model to capture features that evaluate a team's effectiveness in key areas such as aerial duels, set pieces, passing, possession, tackling, and shooting. In the case where calculating the feature would mean dividing by zero, we set the feature value to 0. We create a total of 16 features in this manner, 8 for each team in a match. The list of features created for each match is shown in Table 3.2.

It is also worth mentioning that features that could possibly capture the potential trend of team performance over time were not included for simplicity.

Table 3.2: Overview of created performance metric features (Author’s calculation)

Feature name	Description
aerialSuccessRate_home/away	Ratio of aerals won to total aerals.
cornerAccuracy_home/away	Ratio of accurate corner kicks to total corner kicks.
passAccuracy_home/away	Ratio of accurate passes to total passes.
passKeyPercentage_home/away	Ratio of key passes to total passes.
possessionPercentage_home/away	percentage of total possession time.
tacklesSuccessRate_home/away	Ratio of successful tackles to total tackles.
throwInsAccuracyPercentage_home/away	Ratio of accurate throw-ins to total throw-ins.
shotsOnTargetPercentage_home/away	Ratio of shots on target to total shots.

3.2.2 Historical performance

When predicting the outcome of a future match, we do not have the match statistics available as the match hasn’t occurred yet. For this reason, we have to estimate the performance of each team based on its historical matches.

For every match, we create features that capture historical performance in the following way. For the home(away) team, we calculate the exponentially weighted mean of a specific statistic or performance metric in matches where the home(away) team played on the home(away) side. We do this for every match and every statistic/performance metric in the dataset, for a total of 66 features. The reason we respect the side on which the team played is due to the existence of the home advantage, see Nevill and Holder (1999). The parameter α was set to 0.13. Using a time series split validation and an XGBoost model with default parameters; we tested α values ranging from 0.005 to 0.5 in increments of 0.005. The model trained on features computed with $\alpha = 0.13$ achieved the highest mean accuracy on the validation sets. We will refer to this set of features as *lagged features* onwards.

We also make sure that before every match, there are at least five historical matches with the identical home(away) team played in our dataset so that we do not end up with matches where the calculated features were determined only by a single previous match. Because the Premier League works on the principle of relegation and promotion between English Football League, there are teams that sometimes get promoted to Premier League (‘Premier League’, 2024). Such a case happened, for example, in the season 2023/2024, when Luton Town was promoted into the Premier League after 31 years of not being present in it (‘2023–24 Premier League’, 2024). After we exclude all the matches that do not have at least five reference matches in order to calculate the *lagged features*, 200 matches in total, we are left with 2456 matches in the final dataset.

Exponentially weighted moving average is used to estimate the performance of a team based on historical matches. The idea is to assign higher weights to more recent matches as they are more relevant than older matches. The weight of match i can be computed in the following way:

$$w_i = (1 - \alpha)^i,$$

where α is a parameter that determines how quickly the weights decrease, and $i = 1$ is the index of the most recent match played by the same team (on the same side) before a specific match ('Polars', 2024).

3.2.3 Differences and ratios

Furthermore, in order to capture more complex relationships between the performance of the two teams in a matchup, for every *lagged feature* that we have, we calculate the difference and ratio of the home value to the away value. An example could be the difference between average historic home goals and away goals, which, if close to zero, can help us identify possible draws or, in case of extreme values, identify favourites. This gives us another 66 features. If computing the ratio would result in a division by zero, we replace the feature value with 0.

4. Modeling

4.1 Experiment design

The dataset is first split into training and test data. The test data consists of the Premier League season 2023/2024 and consists of 370 matches (approx. 15%). Even though the season has a total of 380 matches played, ten matches were excluded due to insufficient reference matches needed to calculate the features, as described in section 3.2.2. The training set is further divided into five training/validation splits using a time series split, which is necessary to respect the temporal nature of football matches. The splitting process is explained in section 2.8.3. Each validation set consists of 347 football matches, and training sets consist of 351, 698, 1045, 1392, and 1739 matches in splits 1-5, respectively. These validation splits are used to tune the model hyperparameters. Accuracy is our metric of choice for evaluating hyperparameter combinations on the validation sets, as we do not have a very significant class imbalance between the three classes, and it was also used in existing research. The best hyperparameter combination is chosen based on the highest mean accuracy across the validation sets. Hyperparameters for all models are optimized using random search (see section 2.8.2) with 100 iterations, adding up to 500 model fits, 100 for each data split.

4.2 Feature selection

We have a total of 134 features, 66 features of which are exponentially weighted means of home/away match statistics, Thirty-three are differences between the exponentially weighted means of all match statistics/performance metrics, and 33 are ratios of those features (see more in sec. 3.2.3). The remaining two features are the average ages of the home and away teams.

Not all features might be relevant, so we use Recursive Feature Elimination with Cross Validation (RFECV) to remove the ones that are not important. For RFECV, the estimator used is an XGBoost model with hyperparameters `n_estimators = 50` and `max_depth = 2`, as with default hyperparameters, the model was overfitting heavily to the training sets. After RFECV, the highest mean accuracy on the validation sets is achieved using a combination of 52 features. This selected set of features is further used to train all models.

4.3 Random forest

The hyperparameter space from which we sample 100 random hyperparameter combinations for tuning can be seen in Table 4.1. Out of the sampled hyperparameter combinations, the best one achieved a mean validation accuracy of 54.0%.

Table 4.1: Parameter grid used for hyperparameter tuning of random forest, best hyperparameters are underlined (Author’s calculation)

Hyperparameter	Value
n_estimators	25, 50, 75, <u>100</u> , 125, 150, 175, 200
max_features	'sqrt', 'log2', 0.1, 0.3, 0.5, 0.7, 0.9, <u>1</u>
max_depth	<u>Not limited</u> , 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
min_samples_split	<u>2</u> , 3, 5, 7, 10, 15
min_samples_leaf	1, 2, 3, 4, 5, 6, <u>7</u> , 8
bootstrap	<u>True</u> , False

The training accuracy of the Random Forest algorithm was 73.7%, and the final accuracy on the out-of-sample season 2023/2024 was 58.4%. Although the accuracy numbers look promising at first glance, when comparing them to results obtained by other authors mentioned in Section 1, the model fails to predict any draws and only predicts home/away outcomes. This can be seen in the confusion matrix in Figure 4.1. The most important feature is the absolute difference between the exponentially weighted mean of key passes. Other important features are based on differences/ratios of expectedGoals(xG), ratings and shots. All the features ordered by their importance within the model can be seen in Figure 4.2.

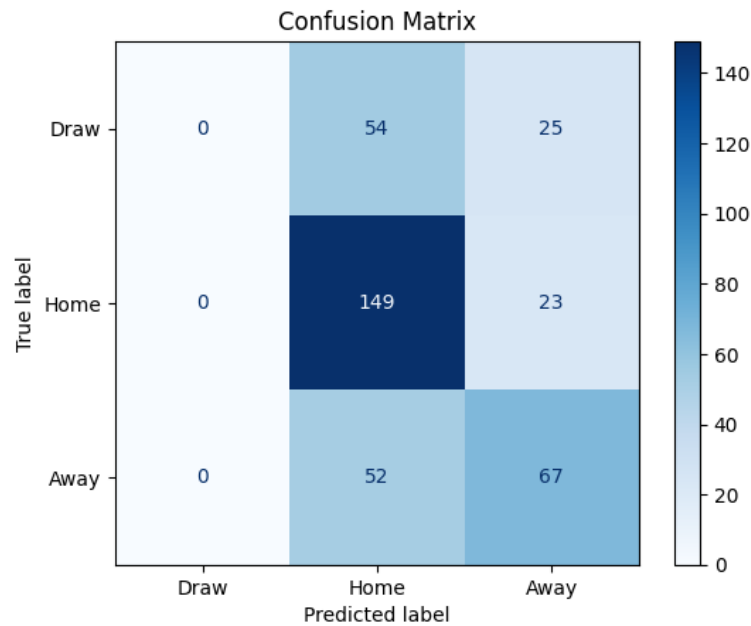


Figure 4.1: Confusion matrix for random forest predictions (Author’s calculation)

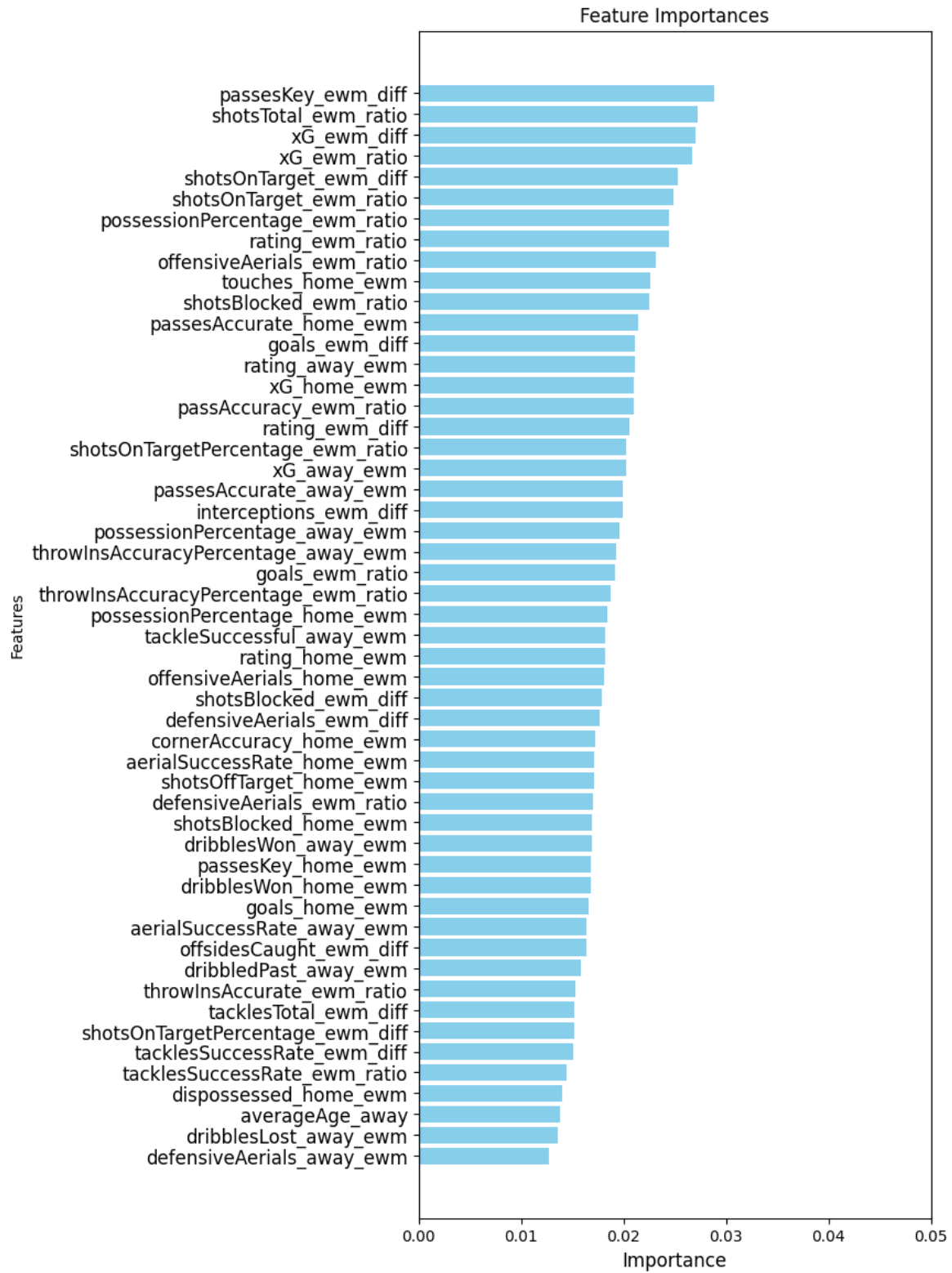


Figure 4.2: Feature importances for random forest (Author's calculation)

4.4 XGBoost

The hyperparameter grid used to tune the hyperparameters of the XGBoost model is shown in Table 4.2. We have decided to tune the key features such as the number of trees, maximum depth of individual trees, training set subsample percentage, and regularization parameters λ and α . A more extensive search could potentially lead to overfitting to the validation dataset (see Levesque (2018)). The regularization hyperparameters α ¹ and λ are tuned to further reduce the risk of overfitting to the training/validation dataset.

With the hyperparameters found using random search, we achieved an accuracy of 54.1% on the training set, 54.4% on the validation set and 58.6% on the testing set. XGBoost, similarly to Random forest, also fails at predicting any draws, as we can see in Figure 4.3. The feature importances determined by XGBoost are very similar to Random Forest, with differences/ratios between the number of key passes, expectedGoals(xG), and shots having the highest importance (see Figure A.1).

Table 4.2: Parameter grid used for hyperparameter tuning of XGBoost, best hyperparameters are underlined (Author’s calculation)

Hyperparameter	Value
max_depth	<u>2</u> , 3, 4
n_estimators	25, <u>50</u> , 75, 100, 125, 150
learning_rate	<u>0.001</u> , 0.01, 0.02, 0.03, 0.05
subsample	<u>0.1</u> , 0.3, 0.5, 0.7, 0.9
lambda	0.01, 0.1, 0.3, <u>0.5</u> , 1, 3, 5
alpha	0.01, 0.1, 0.5, <u>1</u> , 3, 5, 10, 15, 20

¹The hyperparameter α is not related in any way to α mentioned in Section 3.2.2

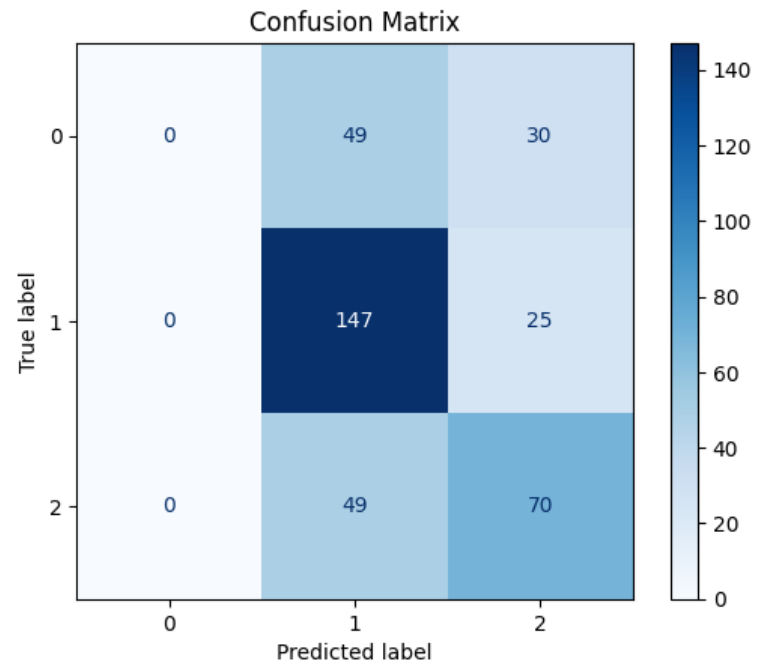


Figure 4.3: Confusion matrix for XGBoost predictions (Author's calculation)

5. Comparison of prediction methods

The best-trained models, XGBoost and Random Forest, achieve very similar performance in terms of accuracy on the test set. We also evaluated the accuracy of the bookmaker, where the prediction was determined by the outcome with the lowest odds offered for simplicity.¹ The accuracy of the machine learning models is comparable to that of a bookmaker on out-of-sample data, although both XGBoost and Random Forest fail to predict any draws. The random forest has much higher accuracy on the training dataset than the XGBoost model, which indicates overfitting. Still, even though it is possibly overfitted, the out-of-sample test accuracy is very similar to XGBoost.

Higher validation and testing accuracy compared to the training for XGBoost is a very interesting occurrence, as this does not happen often in practice. This could theoretically happen due to the usage of regularization hyperparameters *alpha* and *lambda*, which penalize model complexity and are used to prevent overfitting. Other possibilities are that the difference is there due to the sample size of the training set, causing lower train accuracy due to the training set having many more observations than validation and test, or it could be that the model itself is underfitted. It might also be combination of all the factors mentioned above.

Even though the performance of XGBoost and Random Forest on the test set is almost identical, with XGBoost predicting one more match correctly, it's interesting to see that the predictions were not identical for all the other matches. In 20 matches, XGBoost and Random Forest predictions were different, where if one model predicted home, the other predicted away.

In order to address the issue of the models not predicting any draws, undersampling of home and away matches was performed in the training dataset, after which the models were able to predict draws, albeit not with very much success. The model's accuracy also heavily suffered, so this approach was not considered, but different techniques to address this issue are worth exploring in future research.

Table 5.1: Comparison of accuracy on different data splits (Author's calculation)

Model	Training	Validation	Test
Random guess	33.5%	32.8%	32.2%
Always Home	44.7%	44.8%	46.5%
Bookmaker	55.7%	56.1%	58.9%
Random Forest	73.7%	54.0%	58.4%
XGBoost	54.1%	54.4%	58.6%

¹It's important to keep in mind that the lowest odds might not necessarily be the bookmaker's real prediction of outcome, as the bookmakers often adjust the odds based on various factors, such as people predominantly wagering on one outcome. For more details, see 'How Do Bookmakers Set Their Odds?' (2024).

6. Betting simulation

The potential returns if we place bets according to various prediction methods can be seen In Table 6.1. Random Forest predictions achieve the highest return, with XGBoost being very close to it. The evolution of cumulative profit for the entire season 2023/2024 over time for each betting strategy can be seen in Figure 6.1.

Using a non-parametric bootstrap, we also tested whether the total profit θ is statistically significant at a 5% level. For each prediction method, we resampled the test dataset 500 000 times with replacement and computed the statistic total profit $\hat{\theta}_i^*$ for every resample. Non-parametric bootstrap also requires that the statistic is resampled under the null hypothesis($H_0 : \theta = 0$). This requirement can be satisfied by resampling the statistic $\hat{\theta}_i^* - \hat{\theta}$ instead of $\hat{\theta}_i^*$, where $\hat{\theta}$ represents the statistic computed from the original test dataset (see Hall and Wilson (1991)).

Right-tailed P-values were approximated using the bootstrapped distribution(see Chapter 4.4 in Davison and Hinkley (1997)). As no prediction method has a greater p-value than 0.05, we fail to achieve significant profit at the 5% significance level with any of the techniques used. The estimated bootstrap distribution of total profits under the null hypothesis for random forest can be seen in Figure 6.2

Table 6.1: Evaluation of profitability for various prediction methods (Author's calculation)

Model	Total bet	Return	Total Profit($\hat{\theta}$)	$\widehat{\text{P-value}}$
Random guess	37 000	31 655	−5345	0.969
Always Home	37 000	35 215	−1785	0.771
Bookmaker	37 000	36 993	−7	0.506
Random forest	37 000	38 222	1222	0.261
XGBoost	37 000	38 189	1180	0.263

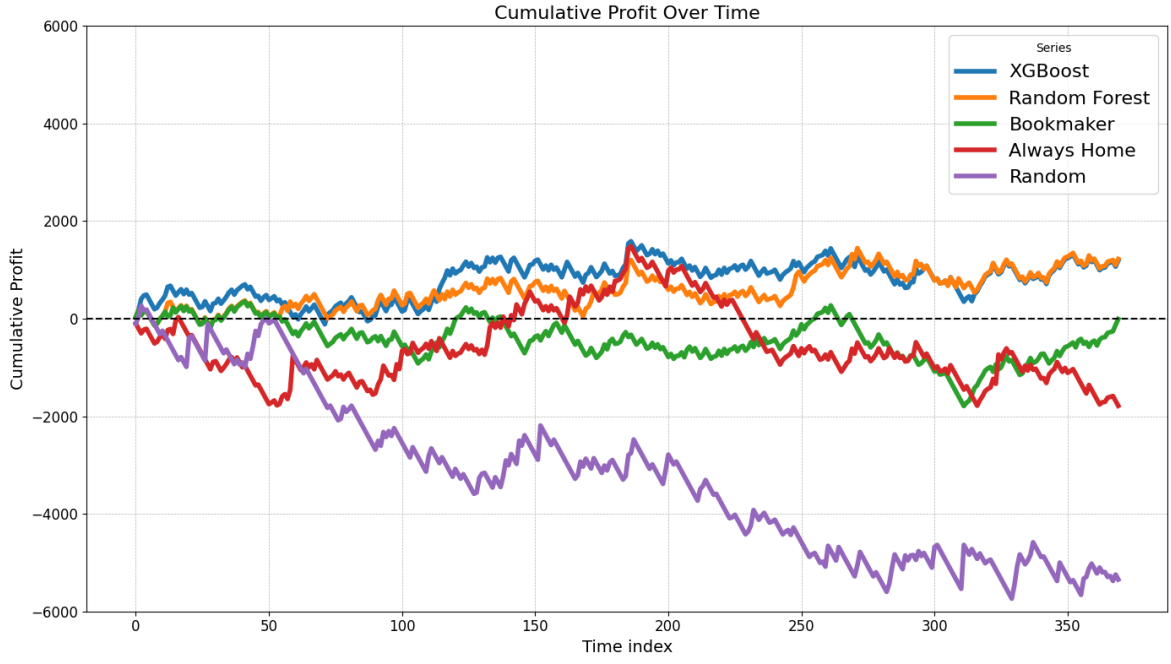


Figure 6.1: Cumulative profit over time for test dataset (Author's calculation)

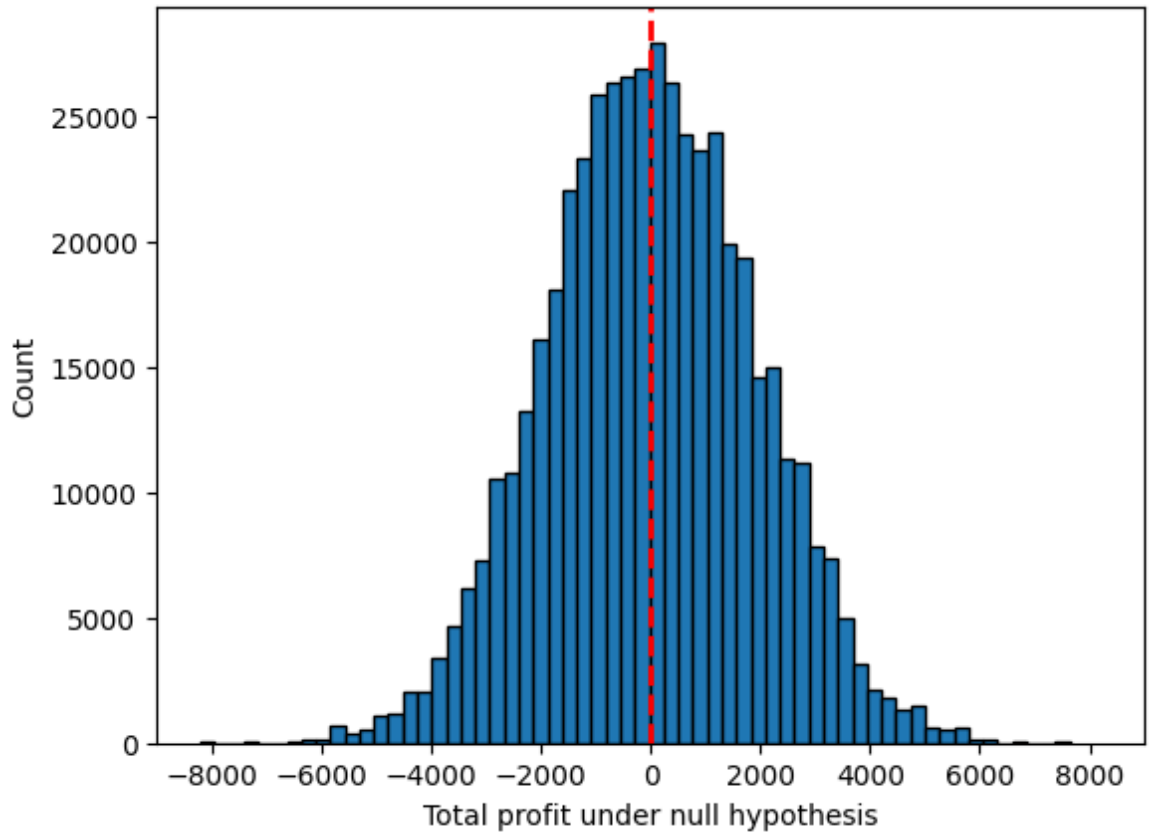


Figure 6.2: Distribution of the bootstrapped statistic $\hat{\theta}_i^* - \hat{\theta}$ for random forest under $H_0 : \theta = 0$ (Author's calculation)

Conclusion

In this bachelor's thesis, we focused on applying machine learning methods for football match prediction. The main goal was to train a model to predict matches as accurately as possible and compare its predictive performance to the bookmaker's accuracy. A secondary goal was to evaluate the potential profitability of our models.

Our work began with a comprehensive literature review, which revealed that most existing studies predominantly used the same dataset with a limited amount of match statistics. We built on this by using a dataset sourced from publicly available data that contained much more detailed attributes and statistics, including expected goals (xG), and to potentially enhance predictive accuracy. Furthermore, we utilized extensive feature engineering to create additional performance metrics and estimate a team's performance based on previous matches. Statistics and metrics from historical matches were aggregated using exponentially weighted mean so that more recent matches have a higher weight.

The dataset contained data from 7 English Premier League(EPL) football seasons, where the last one, 2023/2024, was used as a test set. Out of 134 features, 52 were selected by applying recursive feature elimination with XGBoost as the estimator. Hyperparameters of XGBoost and Random Forest models were tuned using time-based cross-validation splits and random search. Models were then trained on the entire training dataset using hyperparameter combinations that achieved the highest mean validation accuracy.

Random Forest achieved an accuracy of 58.4% on out-of-sample testing data comprised of 370 matches from EPL season 2023/2024. Similarly to Random forest, XGBoost achieved an accuracy of 58.6%. The models achieve a higher accuracy than baseline predictions and come very close to the bookmaker's accuracy, which was 58.9%.

Both models have a significant shortcoming, and that is that they are unable to predict any draws. This issue could be caused by factors such as slight class imbalance or using accuracy as an evaluation metric for the feature selection and hyperparameter tuning. Using different metrics, such as recall or F1-Score, as evaluation metrics is worth exploring in future research to avoid this issue.

We achieved profitability by betting a constant amount on matches according to the predictions of our models, but it was not statistically significant at the 5% level.

In future research, it might be worth exploring the possibility of incorporating player-level features instead, as relying on match data doesn't consider possible injuries of players. Implementing different, more sophisticated betting strategies is also worth considering, as it could lead to statistically significant profits.

Bibliography

- 2023–24 Premier League. (2024, June 12). In *Wikipedia*. Retrieved June 12, 2024, from https://en.wikipedia.org/w/index.php?title=2023%E2%80%9324_Premier_League&oldid=1228650282
Page Version ID: 1228650282.
- Alfredo, Y. F., & Isa, S. M. (2019). Football Match Prediction with Tree Based Model Classification. *International Journal of Intelligent Systems and Applications*, 11(7), 20. Retrieved June 1, 2024, from <https://www.mecs-press.org/ijisa/ijisa-v11-n7/v11n7-3.html>
- Awad, M., & Fraihat, S. (2023). Recursive Feature Elimination with Cross-Validation with Decision Tree: Feature Selection Method for Machine Learning-Based Intrusion Detection Systems. *Journal of Sensor and Actuator Networks*, 12(5), 67. <https://doi.org/10.3390/jsan12050067>
- Berrar, D. (2018, January 1). Cross-Validation. <https://doi.org/10.1016/B978-0-12-809633-8.20349-X>
- Bouchlaghem, Y., Akhiat, Y., & Amjad, S. (2022). Feature Selection: A Review and Comparative Study. *E3S Web of Conferences*, 351, 01046. <https://doi.org/10.1051/e3sconf/202235101046>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Cerqueira, V., Torgo, L., & Soares, C. (2022, February 11). *Model Selection for Time Series Forecasting: Empirical Analysis of Different Estimators*. arXiv: 2104.00584 [cs, stat]. <https://doi.org/10.48550/arXiv.2104.00584>
- Davison, A. C., & Hinkley, D. V. (1997). *Bootstrap Methods and their Application*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511802843>
- FootballData*. (2024). Retrieved June 9, 2024, from <https://www.football-data.co.uk/data.php>
- FootyStats*. (2024). Retrieved June 9, 2024, from <https://footystats.org/>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Gomes, J., Portela, F., & Santos, M. (2015). Decision Support System for predicting Football Game result. Retrieved May 31, 2024, from <https://inase.org/library/2015/zakynthos/bypaper/COMPUTERS/COMPUTERS-57.pdf>
- Hall, P., & Wilson, S. R. (1991). Two Guidelines for Bootstrap Hypothesis Testing. *Biometrics*, 47(2), 757–762. <https://doi.org/10.2307/2532163>
- Heaton, J. (2016). An Empirical Analysis of Feature Engineering for Predictive Modeling. *SoutheastCon 2016*, 1–6. <https://doi.org/10.1109/SECON.2016.7506650>
- Hossin, M., & M.N, S. (2015). A Review on Evaluation Metrics for Data Classification Evaluations. *International Journal of Data Mining & Knowledge Management Process*, 5, 01–11. <https://doi.org/10.5121/ijdkp.2015.5201>

- How Do Bookmakers Set Their Odds?* (2024). Cheeky Punter. Retrieved June 21, 2024, from <https://www.cheekypunter.com/faq/how-do-bookmakers-set-their-odds/>
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Levesque, J.-C. (2018). Bayesian hyperparameter optimization : Overfitting, ensembles and conditional spaces. Retrieved June 9, 2024, from <https://www.semanticscholar.org/paper/Bayesian-hyperparameter-optimization-%3A-overfitting%2C-Levesque/4fff8cdbe7ce1adb5aa2114b3fcfcbe151a097ef>
- Liu, Q., & Wu, Y. (2012). Supervised Learning. https://doi.org/10.1007/978-1-4419-1428-6_451
- Mahesh, B. (2019, January 1). *Machine Learning Algorithms -A Review*. <https://doi.org/10.21275/ART20203995>
- Maleki, F., Muthukrishnan, N., Ovens, K., Md, C., & Forghani, R. (2020). Machine Learning Algorithm Validation. *Neuroimaging Clinics of North America*, 30, 433–445. <https://doi.org/10.1016/j.nic.2020.08.004>
- Naeem, S., Ali, A., Anam, S., & Ahmed, M. (2023). An Unsupervised Machine Learning Algorithms: Comprehensive Review. *IJCDS Journal*, 13, 911–921. <https://doi.org/10.12785/ijcds/130172>
- Nassar, O. (2023, March 10). *Data Leakage in Machine Learning*. <https://doi.org/10.13140/RG.2.2.27468.59528>
- Nevill, A., & Holder, R. (1999). Home advantage in sport: An overview of studies on the advantage of playing at home. *Sports medicine (Auckland, N.Z.)*, 28, 221–36.
- Packt. (2019, May 6). *Cross-Validation strategies for Time Series forecasting [Tutorial]*. Packt Hub. Retrieved June 9, 2024, from <https://hub.packtpub.com/cross-validation-strategies-for-time-series-forecasting-tutorial/>
- Polars. (2024). Retrieved June 17, 2024, from https://docs.pola.rs/api/python/stable/reference/expressions/api/polars.Expr.ewm_mean.html
- Prasetio, D., & Harlili, Dra. (2016). Predicting football match results with logistic regression. *2016 International Conference On Advanced Informatics: Concepts, Theory And Application (ICAICTA)*, 1–5. <https://doi.org/10.1109/ICAICTA.2016.7803111>
- Premier League. (2024, June 10). In *Wikipedia*. Retrieved June 12, 2024, from https://en.wikipedia.org/w/index.php?title=Premier_League&oldid=1228261147
Page Version ID: 1228261147.
- Rawat, T., & Khemchandani, V. (2019). Feature Engineering (FE) Tools and Techniques for Better Classification Performance. <https://doi.org/10.21172/ijiet.82.024>
- RFECV. (2024). scikit-learn. Retrieved June 16, 2024, from https://scikit-learn/stable/modules/generated/sklearn.feature_selection.RFECV.html
- Rokach, L., & Maimon, O. (2005, January 1). Decision Trees. In *The Data Mining and Knowledge Discovery Handbook* (pp. 165–192, Vol. 6). https://doi.org/10.1007/0-387-25465-X_9
- Shin, J., & Gasparyan, R. (2014). A novel way to Soccer Match Prediction. Retrieved June 1, 2024, from <https://www.semanticscholar.org/paper/A-novel-way-to-Soccer-Match-Prediction-Shin-Gasparyan/ccfbc3c712fa025080c5d374174c8a7d5394e9d6>

- Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning*, 5(2), 197–227. <https://doi.org/10.1007/BF00116037>
- Schapire, R. E. (2003). The Boosting Approach to Machine Learning: An Overview. In D. D. Denison, M. H. Hansen, C. C. Holmes, B. Mallick, & B. Yu (Eds.), *Nonlinear Estimation and Classification* (pp. 149–171). Springer. https://doi.org/10.1007/978-0-387-21579-2_9
- Simon, S., Kolyada, N., Akiki, C., Potthast, M., Stein, B., & Siegmund, N. (2023). Exploring Hyperparameter Usage and Tuning in Machine Learning Research. *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 68–79. <https://doi.org/10.1109/CAIN58948.2023.00016>
- Sirakorn. (2020, January 14). *English: Illustration of a boosting method for ensemble learning*. Retrieved June 4, 2024, from https://commons.wikimedia.org/wiki/File:Ensemble_Boosting.svg
- Valentini, G., & Masulli, F. (2002). Ensembles of Learning Machines. *2486*, 3–22. https://doi.org/10.1007/3-540-45808-5_1
- Whitmore, J. (2023, August 8). *What Is Expected Goals (xG)? | Opta Analyst*. The Analyst. Retrieved June 21, 2024, from <https://theanalyst.com/2023/08/what-is-expected-goals-xg/>
- WhoScored. (2024). Retrieved June 9, 2024, from <https://www.whoscored.com/>
- WhoScored-PremierLeague. (2024). Retrieved June 1, 2024, from <https://www.whoscored.com/Regions/252/Tournaments/2/Seasons/5826/England-Premier-League>
- XGBoost. (2024, May 21). In *Wikipedia*. Retrieved June 7, 2024, from https://en.wikipedia.org/w/index.php?title=XGBoost&oldid=1224994987#cite_note-history-12
Page Version ID: 1224994987.
- XGBoost Parameters. (2024). Retrieved June 27, 2024, from <https://xgboost.readthedocs.io/en/stable/parameter.html#>
- Yadav, P. (2019, September 23). *Decision Tree in Machine Learning*. Medium. Retrieved June 23, 2024, from <https://towardsdatascience.com/decision-tree-in-machine-learning-e380942a4c96>

Appendices

A. Full form

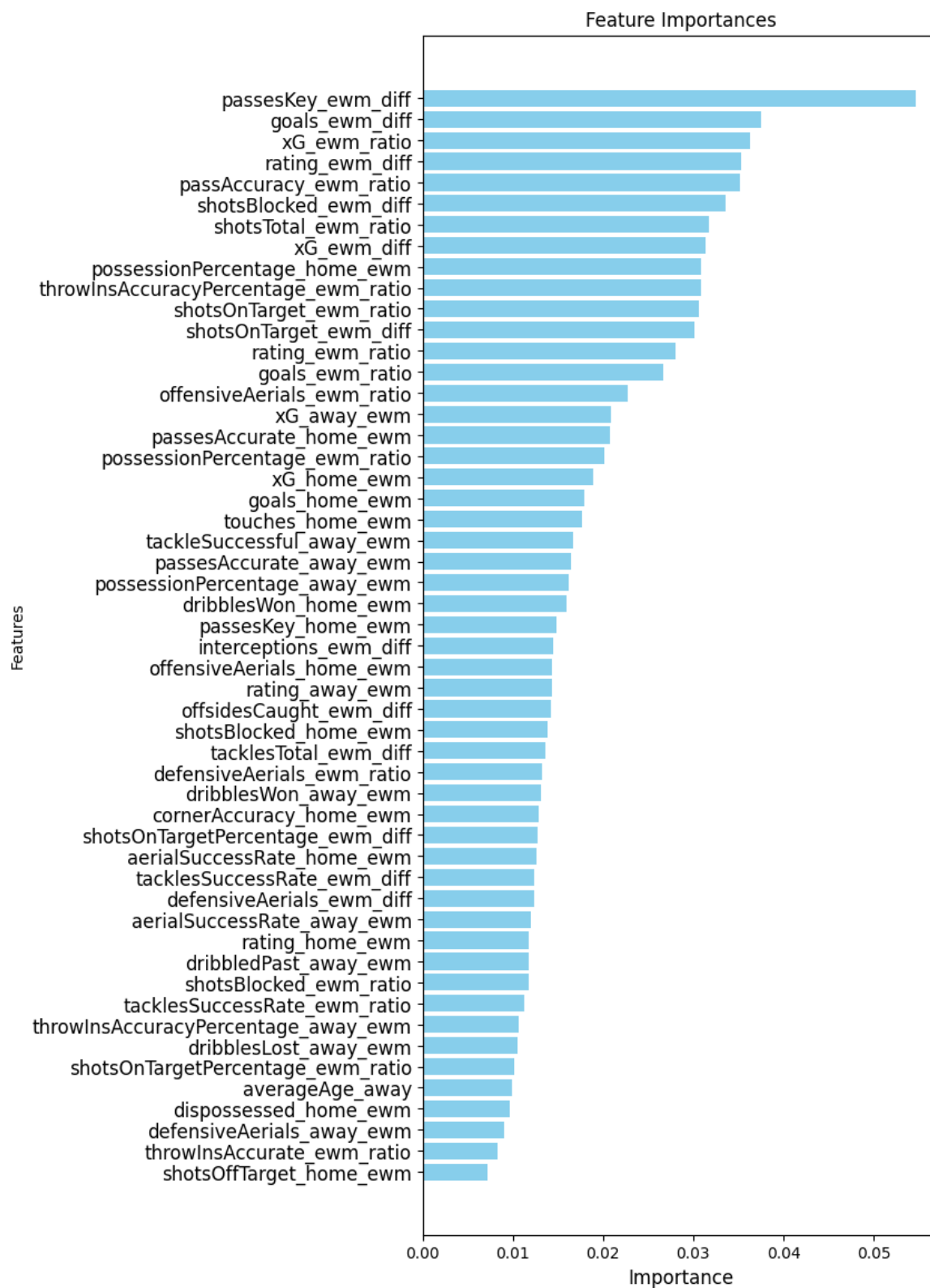


Figure A.1: Feature importances for XGBoost (Author's calculation)